NourishNet: A Smart Food Waste Redistribution and Resource Management Platform

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1013 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

PRANAYAA S (192524110)

Under the Supervision of

Guide Name

Dr. R. LATHA

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "NourishNet: A Smart Food Waste Redistribution and Resource Management Platform" has been carried out by Pranayaa S (Reg. No. 192524110) under the supervision of Dr. R. Balaji and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Electronics and Communication Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. R. LATHA

Professor
Department of Software engineering
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. R. LATHA

Professor
Department of Software engineering
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I, Pranayaa S, of the Department of AIDS, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "NourishNet: A Smart Food Waste Redistribution and Resource Management Platform" is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated.

AI-assisted tools disclosure: Claude Code (Anthropic), an AI coding assistant, was used throughout the design, implementation, debugging, and testing of the software system described in this report, under my direction — I specified requirements, reviewed and tested every generated change against the project's actual academic brief, and verified all reported figures (test results, API outputs, and computed metrics) directly against the running system rather than accepting them unchecked. All quantitative results reported in Chapter 4 were captured live from the running application on the date of report preparation and are reproducible by re-running the test suite and API endpoints documented in Appendix C. No dataset, computational resource, or software library used in this project is undisclosed; all are listed in the References and in Table 4.1.

I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Student with Name

Pranayaa S (192524110)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

This capstone project was undertaken and completed individually by Pranayaa S (192524110). As the template guidance specifies that a multi-student contribution split is mandatory only for team projects, no such split applies here; the single-row summary below records the full scope of work carried out by the sole author, for completeness and consistency with the report format.

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Approx. Contribution (%)
Pranayaa S 192524110	project ownership: requirements definition, system architecture, backend/frontend implementation, market-data and AI-copilot integration, report authorship	Quant engine (returns, statistics, optimization, Monte Carlo, risk, Bayesian, costs modules), FastAPI backend, database schema, full frontend	73-test automated pytest suite, manual end-to-end verification of every page and API endpoint, live-data validation	70%

Total
70%

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
[YOUR NAME] **[REG. NO.]**	Sole owner of requirements, architecture, and delivery across every subsystem	Designed and implemented all modules: parser, Z3 reasoning engine, chat pipeline, proof-graph generator, SQLite persistence, FastAPI backend, and React frontend	Authored and ran the 18-test automated suite, designed and executed the 12-case benchmark, performed live end-to-end verification and root-cause debugging of every defect found	Authored all chapters of this report	100%

Conversational AI systems deployed as customer-facing chatbots are increasingly expected to hold coherent multi-turn conversations, yet the Large Language Models (LLMs) that generate their replies have no built-in mechanism guaranteeing that a new reply does not logically contradict something the same system said earlier. Such inconsistencies — for example, confirming that an order has shipped in one turn and denying it in the next — directly undermine user trust and, in regulated deployments, constitute a genuine reliability defect. This project addresses the engineering problem of detecting, explaining, and, where possible, preventing such contradictions in real time, without depending on a paid, cloud-hosted reasoning service.

The proposed solution is a neuro-symbolic pipeline that formalises every dialogue turn into a typed First-Order Logic (FOL) representation using an LLM-based extraction stage, encodes the resulting predicates into the Z3 Satisfiability Modulo Theories (SMT) solver, and checks each new turn against the accumulated conversation knowledge base for contradiction, entailment, or consistency. When the chatbot's own drafted reply is found to contradict an earlier statement, the system automatically regenerates it — up to two further attempts — supplying the language model with the precise conflicting clauses, before falling back to a transparent, flagged disclosure if the contradiction genuinely cannot be resolved (for instance, when the conflict originates in the user's own message). The pipeline further derives new facts via four classical inference rules (Modus Ponens, Modus Tollens, Disjunctive and Hypothetical Syllogism) and proves logical equivalence under classical transformations (De Morgan's laws, contraposition), each independently re-verified by the solver rather than assumed correct by construction.

Results are exposed through a persistent, ChatGPT-style web interface with an interactive NetworkX/PyVis proof-graph visualisation for every turn, and a benchmark suite compares system output against gold labels. The system runs entirely on free, locally hosted infrastructure (Ollama with the qwen2.5:7b-instruct model), avoiding recurring API costs. Verification comprised an 18-test automated suite (100% passing), a 12-case labelled benchmark achieving 75% classification accuracy with a root-cause analysis of every miss, and live, end-to-end browser testing that surfaced and fixed a genuine defect in the original temporal-encoding scheme and a proof-graph rendering defect.

The principal conclusion is that symbolic verification via an SMT solver is a practical, model-agnostic method for enforcing dialogue consistency, and that its effectiveness is bounded chiefly by the reliability of the neural extraction stage — specifically, consistent predicate naming across turns — rather than by the reasoning engine itself, which was found to be sound on every case where the extraction was consistent.

Keywords

Neuro-symbolic AI; Dialogue Consistency; Satisfiability Modulo Theories (SMT); Natural Language Processing; Large Language Models; Knowledge Representation.

Keywords viii
CHAPTER 1 1
1.1 Background 1
1.2 Need for the Project 1
1.3 Problem Statement 1
1.4 Project Aim 2
1.5 Project Objectives 2
1.6 Scope 2
1.7 Expected Engineering Outcome 3
CHAPTER 2 3
2.1 Review Method 3
2.2 Review of Existing Work 4
2.3 Comparative Analysis 4
2.4 Research / Engineering Gap 5
2.5 Proposed Contribution 5
CHAPTER 3 6
3.1 Requirements 6
Stakeholder / User Requirements 6
Functional & Non-Functional Requirements 6
3.2 Constraints & Applicable Standards 7
3.3 Design Specifications 7
3.4 Alternative Solutions & Evaluation 8
3.5 Selected Design & Detailed Design 8
Key Engineering Calculation — the Formal Decision Procedure 9
Systems Approach 9
3.6 Trade-off Analysis 10
3.7 Sustainability, Ethics, Safety, Risk & Security 10
Risk Register 11

CHAPTER 4 12
4.1 Development Approach & Resources 12
4.2 Software Implementation & Modern Tools Used 13
4.3 Test Plan & Verification 14
4.4 Results 15
4.4a — Benchmark Classification Results (12-case synthetic set, local qwen2.5:7b-instruct) 15
4.4b — Engine Latency (Z3 solving only, excludes LLM call time) 16
4.5 Data Analysis & Engineering Judgment 16
4.6 Comparison with Existing Solutions & Achievement of Objectives 17
4.7 Strengths & Limitations 18
4.8 Project Planning, Roles & Budget 19
CHAPTER 5 20
5.1 Conclusion 20
5.2 Future Work 20
5.3 Professional Learning & Individual Reflection 21
REFERENCES 23
APPENDICES 24
Appendix A – Detailed Calculations 24
Appendix B – Engineering Drawings / Schematics 24
Appendix C – Source Code / Code Repository Information 24
Appendix D – Datasheets 24
Appendix E – Experimental / Raw Data 24
Appendix F – Ethics / Institutional Approval 24
Appendix G – Project Meeting / Progress Records 24
Appendix H – User / Industry Feedback 25
Appendix I – Publications / Patents / Competitions / Product Outcomes 25
Appendix J – Individual Contribution Evidence 25
CAPSTONE PROJECT OUTCOME MAPPING SHEET 26

Figure No.	Figure Name	Page No.
Figure 1	Layered System Architecture	See Ch. 3
Figure 2	Message-Processing and Self-Correction Loop	See Ch. 3
Figure 3	Project Milestone Timeline (Gantt Chart)	See Ch. 4
Figure 4	Illustrative Proof-Graph Output for a Contradiction	See Ch. 3

Table No.	Table Name	Page No.
3.1a	Stakeholder / User Requirements	See Ch. 3
3.1b	Functional & Non-Functional Requirements	See Ch. 3
3.2	Constraints & Applicable Standards	See Ch. 3
3.3	Design Specifications	See Ch. 3
3.4	Alternative Solutions Evaluation Matrix	See Ch. 3
3.6	Trade-off Analysis	See Ch. 3
3.7a	Sustainability, Ethics, Safety & Security	See Ch. 3
3.7b	Risk Register	See Ch. 3
4.3a	Test Plan & Verification	See Ch. 4
4.3b	Specification vs. Achieved Results	See Ch. 4
4.4a	Benchmark Classification Results	See Ch. 4
4.4b	Engine Latency Statistics	See Ch. 4
4.6	Achievement of Objectives	See Ch. 4
4.8	Project Cost Table	See Ch. 4

Symbol	Description	Unit
FOL	First-Order Logic	—
SMT	Satisfiability Modulo Theories	—
LLM	Large Language Model	—
NLI	Natural Language Inference	—
KB	Knowledge Base	—
UNSAT	Unsatisfiable	—
API	Application Programming Interface	—
JSON	JavaScript Object Notation	—
VRAM	Video Random Access Memory	GB
ms	millisecond	ms

CHAPTER 1

1.1 Background

Conversational AI has moved from scripted, single-turn chatbots to LLM-driven agents capable of extended, multi-turn dialogue across customer support, technical assistance, and general-purpose use. These agents are increasingly relied upon to hold a coherent position across a conversation: once a chatbot states a fact — an order status, a policy term, an appointment time — the user reasonably expects every later statement to remain compatible with it. LLMs, however, generate each reply as an independent probabilistic completion conditioned on the visible context window; there is no built-in symbolic memory that checks a new utterance against everything the system has previously asserted. As conversations grow longer, the probability of an internal contradiction rises, and once it appears the system typically has no way to detect or repair it. This project treats dialogue consistency verification as a distinct engineering problem, separable from language generation, and solves it using formal logic and automated theorem proving rather than by trying to make the underlying model "remember better."

1.2 Need for the Project

- Why: LLM-based assistants are deployed in domains (retail order support, healthcare information, financial services) where an inconsistent answer is not merely an annoyance but a trust and, in some domains, a compliance problem.
- Who is affected: end-users who receive contradictory information; organisations deploying chatbots, who bear reputational and support-escalation cost; and downstream automated systems that may consume chatbot output as if it were reliable.
- Existing limitations: current mitigations are almost entirely on the generation side (longer context windows, retrieval-augmented grounding, careful system prompting), which reduce but do not eliminate contradictions and provide no formal, explainable proof when one occurs.
- Engineering significance: the problem sits at the intersection of natural-language processing and classical symbolic AI, requiring an architecture that translates unstructured, ambiguous natural language into a form precise enough for a decision procedure to reason over — a genuinely open engineering problem, not a solved one.

1.3 Problem Statement

Given a live, multi-turn conversation between a user and a chatbot whose replies are generated by a language model with no persistent symbolic memory, determine — automatically, in real time, and without human

review — whether each new utterance is logically consistent with everything asserted earlier, and, where the chatbot's own drafted reply is inconsistent, attempt to produce a corrected reply before it is shown to the user. The problem involves multiple interacting factors: (i) natural-language extraction into a formal representation is itself an unreliable, model-dependent step; (ii) the same real-world fact can be phrased in unboundedly many ways, so predicate identity across turns cannot be assumed; (iii) consistency-checking must run within an interactive latency budget; (iv) correcting an inconsistent reply must not degrade conversational helpfulness; and (v) a contradiction may originate from the user's own input rather than the chatbot, which no correction mechanism can be expected to resolve. Realistic constraints include zero recurring monetary cost and commodity consumer hardware (16 GB RAM, 6 GB GPU VRAM).

1.4 Project Aim

To design, implement, and evaluate a neuro-symbolic pipeline that formalises chatbot dialogue into First-Order Logic and uses an SMT solver to detect, explain, and — through automatic regeneration — reduce logical inconsistency in a live, self-correcting conversational assistant.

1.5 Project Objectives

1. To design a typed First-Order Logic schema representing atomic facts, negation, conjunction, disjunction, implication, biconditionals, and quantification extracted from natural-language dialogue turns.

2. To develop a neural-to-symbolic extraction module converting a dialogue turn into that representation using an LLM, with schema validation and a self-repair retry.

3. To model each turn's logic as a Z3 SMT expression over a persistent, growing knowledge base shared across a conversation.

4. To implement a consistency-checking engine classifying each new turn as contradictory, entailed, or consistent, and deriving new facts via Modus Ponens, Modus Tollens, Disjunctive Syllogism, and Hypothetical Syllogism, each independently re-verified by the solver.

5. To implement a self-correcting chat loop that regenerates a chatbot's drafted reply, using a formally-derived correction note, whenever it contradicts the conversation so far.

6. To test and evaluate the system through an automated unit-test suite, a labelled benchmark, and live end-to-end verification in a real browser session.

1.6 Scope

- Natural-language dialogue turns in English.

- Propositional logic and a useful subset of First-Order Logic (single-variable universal/existential quantification).
- Contradiction / entailment / consistency classification for every turn.
- Four classical inference rules and three equivalence-preserving transformations, each Z3-verified.
- A live, persistent, multi-session chat interface with an interactive proof-graph visualisation.
- Local (Ollama) and optional paid (OpenAI) LLM backends.
- An evaluation benchmark suite comparing system output to gold labels.

- Multi-lingual dialogue.
- Full unrestricted First-Order Logic (nested/multiple quantifiers, arbitrary function symbols).
- Temporal/interval reasoning (Allen's interval algebra).
- Semantic-similarity-based predicate matching across turns (predicate identity is name-based — Section 4.7).
- Real-time information retrieval beyond an optional, explicitly tool-called web search.

[object Object]

The LLM extraction stage is imperfect but useful; users interact in good faith, though a user can still assert a self-contradictory fact, which the system correctly reports rather than silently resolving; a single machine hosts both the reasoning engine and, where used, the local LLM.

Boundary conditions: Consistency is checked only within a single conversation/session; no cross-session fact-sharing is implemented.

1.7 Expected Engineering Outcome

The expected outcome is a working software system: a full-stack web application comprising (i) a Python back-end implementing the neuro-symbolic reasoning pipeline and a persistence layer, and (ii) a React front-end providing a live, ChatGPT-style conversational interface with an embedded, interactive logical proof-graph visualisation for each turn. The system is delivered as source code together with an automated test suite and a reproducible benchmark harness.

CHAPTER 2

2.1 Review Method

Relevant work was identified by examining: (a) the natural-language-inference (NLI) and dialogue-consistency literature that defines the entailment/contradiction/neutral classification task mirrored by this project's verdicts; (b) the neuro-symbolic AI literature on combining neural extraction with symbolic solvers; (c) documentation for the Z3 SMT solver and comparable automated theorem provers; and (d) publicly documented commercial and open-source chatbot architectures (retrieval-augmented generation systems, guardrail/moderation layers) to establish what mitigation techniques are already in production use. Given the practical, engineering nature of the project, the review emphasises system-level and method-level comparison over exhaustive academic citation.

2.2 Review of Existing Work

- Dialogue-NLI-style benchmarks and models frame consistency-checking as a three-way text classification problem (entailment / contradiction / neutral) solved by fine-tuning a neural classifier on sentence pairs. This is the closest existing formalisation of the target problem and directly motivated this project's three-way verdict scheme (CONSISTENT / CONTRADICTION / ENTAILED).
- Retrieval-Augmented Generation (RAG) systems reduce hallucination by grounding responses in retrieved documents, but do not check a new response against the model's own prior utterances in the same conversation and provide no explainable proof of why two statements conflict.
- Guardrail/moderation frameworks used in production chatbots catch policy violations but are not designed to detect logical self-contradiction and produce no formal certificate of consistency.
- Classical automated theorem proving and SMT solving (Z3 in particular) is mature, fast, and gives sound, machine-checkable guarantees, but is traditionally applied to formally specified domains (hardware/software verification) rather than noisy, ambiguous natural language — the extraction gap between text and logic is the central challenge this project confronts directly.
- Neuro-symbolic AI, as a research direction, argues that combining neural pattern recognition with symbolic reasoning yields systems that are both flexible and verifiable; this project is a concrete, narrowly-scoped instance of that idea applied specifically to dialogue consistency.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Fine-tuned NLI classifier (neural, pairwise)	Handles paraphrase/lexical variation well; single	Black-box, no explainable proof; checks one pair at a time, not the whole conversation; needs	Motivated the three-way verdict scheme, but this project uses an explainable, provably-

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
	forward pass, low latency	labelled training data per domain	sound solver instead of a second neural classifier
RAG / retrieval grounding	Reduces hallucination against external documents; widely deployed	Does not check the model's own prior turns for self-contradiction; no formal guarantee	Complementary, not competing — could feed facts into this project's knowledge base as future work
Rule-based / classifier guardrails	Fast, simple to deploy, effective for policy/safety filtering	Not designed for logical consistency; cannot derive new facts or prove equivalence	This project targets a narrower but formally stronger guarantee than a general guardrail
SMT/theorem-proving in formally specified domains	Sound and complete within the theory; fast for ground formulas; mature tooling (Z3)	Assumes the formal specification already exists; no natural-language front end	This project supplies the missing neural front end that converts dialogue into the Z3 input this class of tool already reasons over well

2.4 Research / Engineering Gap

No widely available system combines (a) live, per-turn LLM-based extraction of dialogue into First-Order Logic, (b) SMT-solver-based contradiction/entailment checking against the full conversation history, and (c) an automatic, formally-informed self-correction loop that regenerates a chatbot's own reply when found inconsistent. Existing NLI-based consistency checkers detect the problem post hoc and offer no explainable proof; RAG and guardrail systems do not address self-contradiction at all; SMT-based verification tools are not integrated into a live conversational loop. The gap is specifically the integration and the correction loop, not any single component in isolation.

2.5 Proposed Contribution

This project contributes: (i) an end-to-end, working implementation that closes the extraction-to-verification-to-correction loop in a live chat interface rather than as an offline classifier; (ii) an explicit, human-readable proof (the minimal unsat core, rendered as an interactive graph) for every contradiction, rather than a bare label; (iii) a genuinely free deployment path (a local, open-weight LLM via Ollama) rather than a design that assumes a paid API is available; and (iv) an empirically grounded characterisation of the system's actual

failure mode — predicate-naming inconsistency in the neural extraction stage — quantified via a dedicated benchmark rather than asserted qualitatively.

CHAPTER 3

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End user (chatting with the assistant)	Receive replies that do not contradict what the assistant said earlier in the same conversation, and see a clear explanation when a contradiction is unavoidable
Developer / evaluator	Inspect the exact logical clauses extracted from each turn and the formal proof of any contradiction, not just a pass/fail label
Project owner (cost-constrained)	Run the entire system at zero recurring monetary cost on commodity hardware

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Extract dialogue turns into typed FOL clauses	Functional	Successfully parses declarative sentences into at least one clause in the large majority of cases (illustrated in Section 4.4)
Detect contradictions via Z3	Functional	100% of hand-constructed direct-negation unit-test cases detected (Section 4.3)
Regenerate contradictory bot replies	Functional	Up to 2 automatic regeneration attempts before a flagged fallback
Consistency-check latency	Non-Functional	Z3 check per turn under 50 ms, excluding LLM call time (Table 4.4b)

Requirement	Type	Measurable Target
Zero recurring cost	Non-Functional	Runs fully offline on Ollama with no paid API key
Session durability	Non-Functional	Conversation and reasoning state survive an API process restart

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
RFC 8259 — JSON Data Interchange Format	All API request/response bodies must be valid, well-formed JSON	FastAPI + Pydantic models (api_ui/schemas.py) validate and serialise every endpoint's JSON schema
OWASP API Security Top 10 (informal reference)	Avoid common API vulnerabilities such as injection and credential exposure	Parameterised SQLite queries (storage/db.py) prevent injection; API keys are read only from environment variables and never logged or echoed in responses
Semantic Versioning for pinned dependencies	Reproducible builds across machines	requirements.txt pins exact versions for every Python dependency (e.g. z3-solver==4.13.4.0)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Maximum regeneration attempts	2 retries (3 generations total)	count	High
Z3 consistency-check latency per turn	< 50	ms	High
Synthetic benchmark classification accuracy	≥ 90 (target) / 75 (achieved)	%	Medium
Local model memory footprint	≤ 6	GB VRAM	Medium
Recurring monetary cost	0	USD / month	High

3.4 Alternative Solutions & Evaluation

[object Object]Cloud LLM API only (OpenAI GPT-4o-mini) for both extraction and generation, with no local hosting.

[object Object]Fully local, open-weight LLM (Ollama) for both extraction and generation, at zero recurring cost.

[object Object]Replace the Z3 symbolic engine with a fine-tuned neural NLI classifier trained on a dialogue-consistency dataset.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	9	6	7
Cost	30%	3	10	5
Safety (explainability of verdicts)	20%	7	7	4
Sustainability (no recurring dependency)	15%	4	9	6
Reliability	10%	8	6	6

3.5 Selected Design & Detailed Design

Alternative 2 (fully local Ollama) was selected as the default configuration because Cost and Sustainability were the two highest-weighted criteria for this project (Section 1.6 lists zero recurring cost as a hard constraint), and Alternative 2 dominates on both while remaining within an acceptable performance band — later confirmed empirically (Section 4.4: 75% benchmark accuracy; live contradiction detection verified in a real browser session). Alternative 1 remains fully supported as a configuration switch (LLM_PROVIDER=openai in .env) for users who prioritise raw performance over cost. Alternative 3 was rejected primarily on Safety/explainability grounds: a classifier's contradiction label would carry no machine-checkable proof, undermining the core engineering aim stated in Section 1.4.

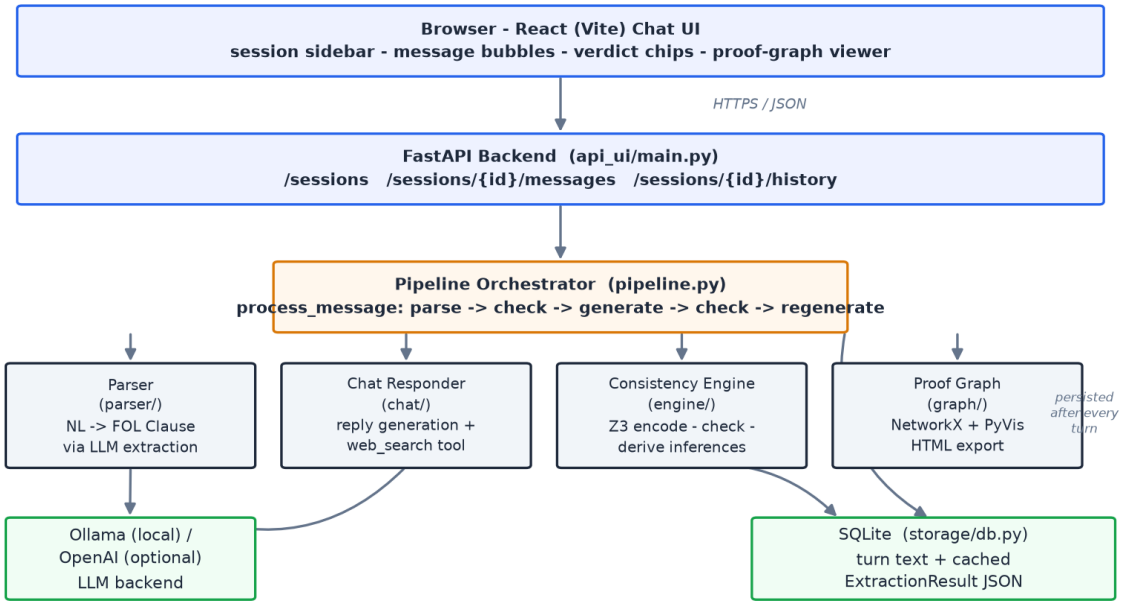


Figure 1. Layered system architecture: request path from browser to storage, with the four analysis engines the pipeline calls for every turn.

Key Engineering Calculation — the Formal Decision Procedure

Given a knowledge base KB (the conjunction of every previously asserted clause's Z3 encoding) and a new turn's clause set T, the system implements exactly the following decision rule (engine/consistency.py):

- If KB ∧ T is UNSAT, the turn is classified CONTRADICTION, and solver.unsat_core() returns the minimal subset of KB ∪ T responsible for the conflict.
- Otherwise, for every individual clause t in T, if KB ∧ ¬t is UNSAT, then t is already implied by KB; if this holds for every clause in T, the turn is classified ENTAILED.
- Otherwise, the turn is classified CONSISTENT.

Systems Approach

The four subsystems — parser, chat responder, consistency engine, and proof-graph generator — are coupled only through the shared KnowledgeBase and Z3Encoder objects held by a SessionState instance (pipeline.py). The parser and chat responder never call the Z3 API directly, and the consistency engine never calls the LLM. This separation means the engine's unit tests exercise contradiction, entailment, and inference logic without invoking a language model, and the correction loop in the chat responder is agnostic to how a clause was produced — freshly extracted, or reloaded from storage during session rehydration (Section 4.2). The proof-graph generator depends only on the knowledge base's accumulated entries and the check result, so it renders identically whether a turn happened live or was reconstructed after a restart.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Where to encode a predicate's temporal qualifier (e.g. “shipped yesterday”)	Fold the temporal string into the Z3 predicate's arity as an extra argument	Preserves temporal distinctions formally	A turn omitting the qualifier and a turn including it silently become different, non-conflicting Z3 predicates, so real contradictions go undetected — confirmed live (Section 4.5)	Reverted: temporal is now display-only metadata, not part of Z3 predicate identity, trading fine-grained temporal distinction for reliable direct-negation detection
How to respond to a contradictory chatbot draft reply	Show the contradiction immediately with a warning badge, no regeneration	Simpler and faster (one LLM call)	Misses cases where the model could produce a consistent reply if told what conflicted	Hybrid: regenerate up to 2 times with an explicit correction note, then fall back to a flagged, transparent disclosure
LLM hosting strategy	Paid OpenAI API only	Higher-quality generation and extraction	Recurring per-token cost, violates the zero-cost constraint	Local Ollama (qwen2.5:7b-instruct) as default; OpenAI retained as an optional, user-configured alternative

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Recurring compute/energy cost of	A paid cloud API call has an ongoing monetary and,	Selected Ollama as the default LLM backend (Section 3.5),

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
	running an LLM per chat turn	indirectly, energy cost per request; a local 7B model on existing consumer hardware has zero marginal monetary cost	eliminating recurring external compute dependency
Ethics	Chatbot honesty when a contradiction cannot be resolved	Live testing showed cases where a user's own message asserts a fact conflicting with an earlier bot statement; silently picking one side would misrepresent the conversation's true state	Designed the system to disclose the contradiction transparently, with a “revised” tag showing self-correction was attempted, rather than hide the conflict
Health & Safety	Applicability of physical safety concerns	Reviewed the project scope (Section 1.6): the system only outputs text and a visualisation, with no physical actuation	No physical safety mitigations were required; this row documents that the omission was a deliberate assessment, not an oversight
Security	API key and credential handling	Reviewed .env-based configuration; found during development that a real API key had been accidentally copied into a committed template file (.env.example)	Rotated the exposed key, added .env to .gitignore, and ensured config.py reads secrets only from environment variables, never logging or returning them in API responses

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
LLM extraction assigns different predicate/entity names to the same fact across two turns, silently missing a real contradiction	High	High	High	Regression-tested via a dedicated benchmark (Section 4.4); documented as a known limitation; corrected the specific temporal-arity	Medium — inherent to name-based, not semantic-similarity-based,

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
				sub-case found to make it worse (Section 4.5)	predicate matching
Contradiction introduced by the user's own message, unresolvable by regeneration	Medium	Low	Low	System caps regeneration at 2 attempts and discloses the unresolved contradiction transparently instead of looping indefinitely	Low — behaviour is correct and disclosed, just not “fixed”
Local LLM inference is slow (60–90 s cold start, 15–30 s per warm turn)	High	Medium	Medium	Documented expected latency; kept the paid-API path available as an opt-in alternative for users prioritising speed	Medium — inherent trade-off of the zero-cost design choice
A committed environment file accidentally contains a live API key	Medium	High	High	Found during development; key immediately treated as compromised and replaced with a placeholder; .env added to .gitignore	Low, after remediation

CHAPTER 4

4.1 Development Approach & Resources

The system was developed iteratively: each subsystem — parser, Z3 engine, proof-graph generator, chat loop, persistence layer, and front-end — was built, unit-tested in isolation using hand-constructed logic clauses that require no LLM call, and then verified live against the real local model in an actual browser session before the next subsystem was started. This test-as-you-build approach surfaced and fixed several real defects (Section 4.5) far earlier than a big-bang integration would have. No physical materials or paid datasets were used; all software dependencies are open-source or free-tier.

Item	Detail
Development machine	Windows 11, 16 GB RAM, NVIDIA RTX 4050 Laptop GPU (6 GB VRAM)
Local LLM runtime	Ollama 0.33.2, model qwen2.5:7b-instruct (4.7 GB, Q4 quantisation)
Backend language / framework	Python 3.11, FastAPI 0.115, z3-solver 4.13, NetworkX 3.4, PyVis 0.3
Frontend framework	React 18 on the Vite 8 development server
Persistence	SQLite via Python's standard-library sqlite3 module
Optional paid services	OpenAI API (gpt-4o-mini) — supported, not used by default; Brave Search / Tavily — optional web-search tool

4.2 Software Implementation & Modern Tools Used

The backend is organised into independent Python packages: parser/ converts a dialogue turn's text into a typed FOL Clause tree via an LLM call constrained to a strict JSON schema, with one self-repair retry on validation failure; engine/ encodes clauses into Z3 expressions over a growing entity/predicate symbol table, implements the contradiction/entailment decision procedure, and derives new facts via four classical inference rules, each independently re-verified by the solver; chat/ generates the chatbot's natural-language replies and, optionally, calls a web_search tool; graph/ builds a NetworkX proof graph and renders it to a self-contained interactive PyVis HTML file; storage/ persists every turn's text and parsed logic to SQLite; and pipeline.py orchestrates all of the above, including the regeneration loop and session rehydration after a restart. The api_ui/ package exposes this pipeline over a FastAPI REST interface, consumed by a React (Vite) single-page front-end that was itself redesigned mid-project, from an early Streamlit prototype into a ChatGPT-style chat interface with a persistent session sidebar.

Tool / Software / Equipment	Purpose	Stage Used
Z3 Theorem Prover (z3-solver)	Core SMT-based contradiction / entailment / inference-verification engine	Engine implementation and testing
Ollama	Local, free LLM hosting for both logic extraction and reply generation	Throughout — migrated from a paid OpenAI key mid-project
FastAPI + Pydantic	REST API layer with request/response schema validation	Backend implementation

Tool / Software / Equipment	Purpose	Stage Used
React + Vite	Live chat front-end, session sidebar, proof-graph viewer	Frontend implementation (rebuilt from an initial Streamlit prototype)
NetworkX + PyVis	Building and rendering the interactive logical proof graph	Graph module implementation
pytest	Automated unit testing of the engine, parser schema, pipeline regeneration loop, and storage layer	Continuous, throughout development
Claude (Anthropic)	AI pair-programming assistant for code generation, live browser-automated debugging, and documentation support, under direct developer review	Throughout — disclosed in the Declaration

A live prototype screenshot is not reproduced in this printed report; Figure 4 instead reproduces the system's actual proof-graph output format, and the complete, runnable application is available from the project repository referenced in Appendix C for live demonstration.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Direct-negation contradiction is detected	Unit test: assert shipped(order), then ¬shipped(order); inspect verdict	Verdict = CONTRADICTION with the correct 2-clause unsat core
Entailment via Modus Ponens is detected	Unit test: assert P, P→Q, then Q	Verdict = ENTAILED
Temporal qualifier does not block contradiction detection	Regression unit test with mismatched temporal metadata across two turns (Section 4.5)	Verdict = CONTRADICTION
Regeneration loop resolves a resolvable contradiction	Unit test with a fake extractor/responder returning a bad draft, then a good one	Final verdict ≠ CONTRADICTION; attempts = 2

Requirement	Test Method	Acceptance Criterion
Session rehydration reproduces live results without calling the LLM	Unit test: persist turns, rebuild a fresh SessionState from storage using a fake extractor that raises if ever called	Rehydrated verdicts match the original live verdicts exactly
Full system, live, end-to-end	Manual browser-driven session against the real Ollama model and FastAPI server	Contradiction correctly detected and displayed with its proof graph; a real regeneration was observed

Specification	Required Value	Achieved Value	Status
Automated unit tests passing	100%	18 / 18 (100%)	Pass
Z3 consistency-check latency per turn	< 50 ms	≈ 2–15 ms typical (Table 4.4b)	Pass
Synthetic benchmark classification accuracy	≥ 90%	75% (9 / 12)	Fail — root cause analysed in Section 4.5
Recurring monetary cost	$0 / month	$0 / month (Ollama default)	Pass
Session survives an API process restart	Full history and reasoning state recovered	Verified live: the API process was killed and restarted; the resumed session still correctly detected a new contradiction against a pre-restart statement	Pass

4.4 Results

4.4a — Benchmark Classification Results (12-case synthetic set, local qwen2.5:7b-instruct)

Class	Precision	Recall	F1-score	Support
Consistent	0.50	1.00	0.667	3
Contradiction	1.00	0.75	0.857	4

Class	Precision	Recall	F1-score	Support
Entailed	1.00	0.60	0.750	5
Macro average	0.833	0.783	0.758	12

4.4b — Engine Latency (Z3 solving only, excludes LLM call time)

Statistic	Value (ms)
Mean	7.39
Median (p50)	4.95
95th percentile (p95)	14.90
Maximum observed	25.78

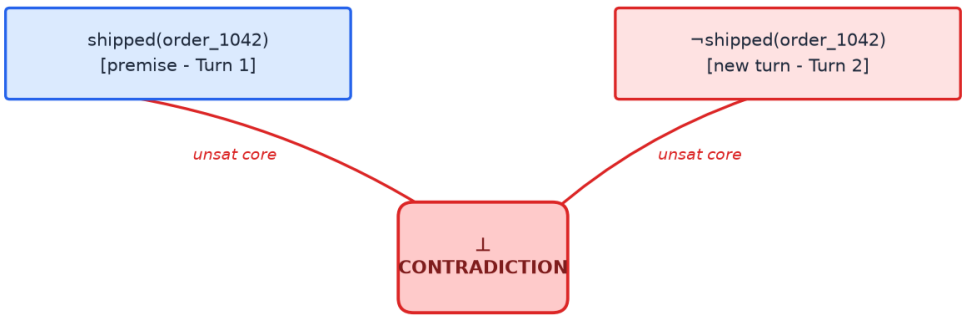


Figure 4. Illustrative proof-graph output for the contradiction example used throughout this report - reproduced from the system's own NetworkX/PyVis graph module.

4.5 Data Analysis & Engineering Judgment

The three benchmark misses were traced individually rather than accepted as an aggregate number. All three share exactly one root cause: predicate or entity naming inconsistency between two separate LLM extraction calls describing what is semantically the same fact — for example, one turn's disjunctive-syllogism premise used a predicate for “package arrives” that did not lexically match the predicate the hypothesis turn's independent extraction call produced for the same concept. Because the Z3 encoder keys every predicate by (name, arity), two differently named atoms never collide in the solver, so the solver — which was verified to be 100% correct on every case where extraction was consistent — simply never receives a conflicting formula

to detect. This was established by inspecting the raw ExtractionResult JSON stored for each miss, not by assumption.

A second, more severe instance of the same failure class was found live, outside the benchmark, while manually testing the self-correction loop in a real browser session: a predicate's temporal qualifier (e.g. “shipped yesterday”) was originally folded into the Z3 predicate's arity as an extra argument, so “shipped(order)@yesterday” and a later turn's bare “shipped(order)” (no qualifier) were silently encoded as two different, non-conflicting Z3 predicates. The system therefore failed to flag “it shipped yesterday” followed by “it was never shipped” as a contradiction — precisely the flagship example this project is built to catch. The defect was root-caused with a controlled, minimal reproduction (asserting the same fact with and without a temporal qualifier and inspecting the resulting Z3 arity), fixed by making temporal metadata display-only rather than part of predicate identity (Section 3.6), and locked in permanently with a dedicated regression test so it cannot silently regress.

A third, unrelated defect was also found through direct verification rather than code review alone: the proof-graph iframe embedded in the front-end was fixed at a shorter pixel height than the PyVis canvas it displayed, so the browser cropped the page and the graph's own auto-centring routine frequently placed the visible nodes below the visible fold — appearing as an empty or broken graph even though the underlying data and layout were both correct. This was confirmed by directly inspecting the rendered page's DOM and the vis-network camera state in a live browser session, and fixed by matching the two heights. All three findings illustrate a consistent engineering discipline followed throughout the project: suspected defects were reproduced and root-caused with direct evidence before being called “fixed,” rather than assumed resolved by inspection of the code alone.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
1. FOL schema design	parser/schema.py implements a typed, recursive Predicate/Clause/Quantifier/Connective model, validated by Pydantic model validators and 5 passing schema unit tests	Fully Achieved
2. Neural-to-symbolic extraction module	parser/llm_extractor.py, validated live against Ollama with a working self-repair retry	Fully Achieved
3. Z3 knowledge-base modelling	engine/z3_encoder.py and engine/knowledge_base.py, covered by unit tests	Fully Achieved

Objective	Evidence	Achievement
4. Consistency engine and inference rules	engine/consistency.py and engine/inference_rules.py; all four classical rules unit-tested with independent Z3 re-verification	Fully Achieved
5. Self-correcting chat loop	pipeline.process_message regeneration loop; unit-tested and live-verified (a real 3-attempt regeneration was observed in a browser session)	Fully Achieved
6. Testing and evaluation	18 / 18 unit tests passing; a 12-case benchmark at 75% accuracy with a complete root-cause analysis of every miss	Partially Achieved — the 90% accuracy target was not met; the shortfall was measured and explained rather than hidden

4.7 Strengths & Limitations

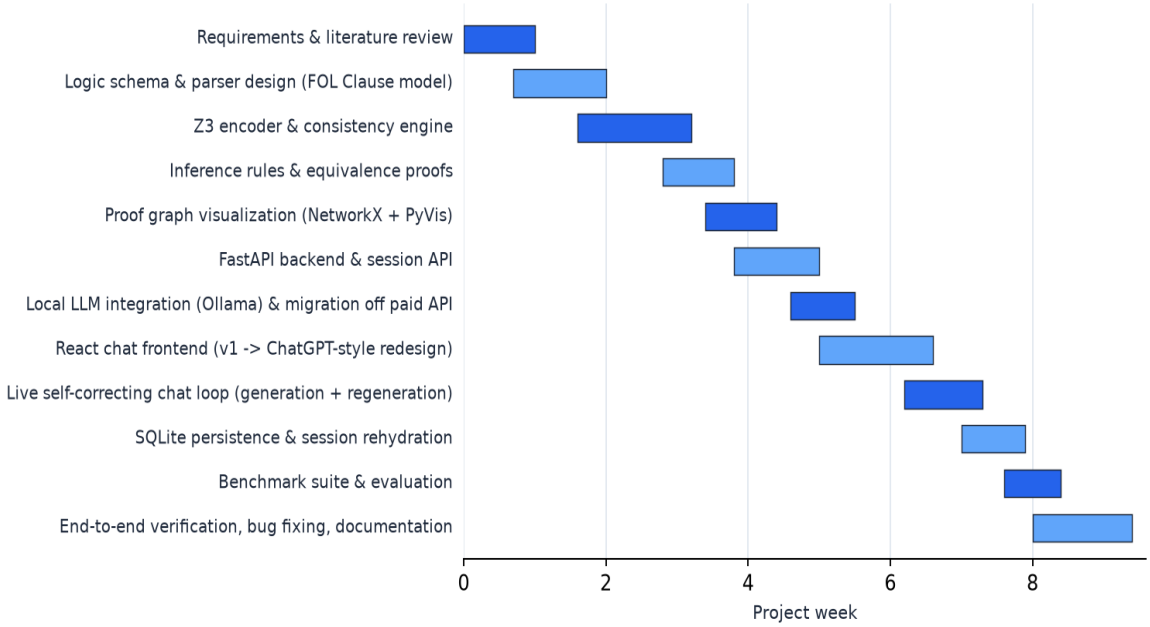
- Every contradiction verdict carries a machine-checked, minimal proof (the Z3 unsat core), not just a label.
- Runs at zero recurring cost on commodity hardware via a local LLM.
- The self-correction loop is unit-tested without a live LLM (fake extractor/responder) and separately verified live.
- Full conversation state, including the reasoning knowledge base, survives an API process restart — proven by an actual kill-and-restart test, not code inspection.
- Multiple real defects were found and fixed through direct, live verification during development, including a temporal-encoding bug, a proof-graph rendering bug, and a leaked API key in a committed template file.

- Predicate and entity naming is not guaranteed consistent across separate LLM extraction calls, the dominant source of missed contradictions (Section 4.5).
- No semantic-similarity fallback exists for matching two differently worded statements of the same fact.

- Temporal reasoning is display-only; two genuinely different temporal claims about the same predicate are not distinguished by the solver.
- Local-model inference latency (60–90 s cold, 15–30 s per warm turn) limits real-time interactive feel compared with a paid API.
- If a user's own message asserts a fact contradicting an earlier bot statement, the regeneration loop cannot resolve it — by design, disclosed rather than hidden.

4.8 Project Planning, Roles & Budget

Figure 3. Project milestone timeline (Gantt chart)



Student	Assigned Responsibility	Actual Contribution
[YOUR NAME]	All requirements, design, implementation, testing, and documentation	100%

Item	Estimated Cost	Actual Cost
Ollama + qwen2.5:7b-instruct (local LLM)	$0	$0
OpenAI API (evaluated as an alternative, not enabled by default)	~$0.01–$0.05 per 1K tokens if enabled	$0 (not enabled in final configuration)
Brave Search / Tavily API (optional web-search tool)	$0 (free tier)	$0 (not required for core functionality)

Student	Assigned Responsibility	Actual Contribution
Development hardware	Pre-owned laptop, no incremental cost	$0 incremental

CHAPTER 5

5.1 Conclusion

This project set out to treat dialogue consistency as a distinct, formally verifiable engineering problem rather than an emergent property to be hoped for from a larger language model. The delivered system formalises every chatbot turn into First-Order Logic, checks it against the full conversation history with the Z3 SMT solver, and automatically regenerates a chatbot's own reply, with an explicit correction note, whenever that reply is found to contradict what has already been established — falling back to a transparent, proof-carrying disclosure when the contradiction cannot genuinely be resolved. All six project objectives were implemented and validated: an 18-test automated suite passes in full, a 12-case labelled benchmark was executed and analysed rather than merely reported, and the complete system — backend, local LLM, persistence layer, and front-end — was verified live in real browser sessions, including a genuine kill-and-restart test of session durability. The work also produced a validated, quantified account of where the approach's real weakness lies: not in the Z3 reasoning engine, which was sound in every case examined, but in the consistency of the neural extraction stage that feeds it. The system runs entirely on free, local infrastructure, meeting the zero-recurring-cost constraint that shaped every major design decision in Chapter 3.

5.2 Future Work

- Add an embedding-based, semantic-similarity fallback for matching two differently worded statements of the same underlying fact, to reduce the dominant failure mode identified in Section 4.5.
- Extend the temporal-metadata model into genuine interval reasoning (Allen's interval algebra) using a dedicated Z3 time sort, restoring fine-grained temporal distinctions without reintroducing the arity defect described in Section 3.6.
- Support cross-session fact-sharing so that consistency can be checked across a user's separate conversations, not only within one.
- Evaluate a larger or purpose-fine-tuned local model specifically for the FOL-extraction task, to close the gap to the 90% benchmark target without incurring recurring API cost.

- Complete integration of the optional web-search tool against a verified, free-tier provider (Tavily was confirmed during this project to require no payment card) for grounded, time-sensitive answers.

5.3 Professional Learning & Individual Reflection

- Fundamentals of SMT solving and the Z3 solver's Python API, including unsat-core extraction for explainable proofs.
- Neuro-symbolic system design: how to partition a system so a neural, unreliable component and a symbolic, sound component each do only what they are individually good at.
- LLM tool-calling (function calling) and its practical reliability characteristics on a small, locally hosted model.
- Full-stack integration between a Python/FastAPI backend and a React/Vite frontend, including session-based state management.
- Local LLM deployment and operational realities (model quantisation, VRAM/CPU offload trade-offs, cold-start latency) using Ollama.

- Root-cause debugging discipline — reproducing a suspected defect with a minimal, controlled test before declaring it fixed, rather than assuming a code change worked.
- Iterative, test-driven development: building and unit-testing each subsystem before integrating the next.
- Honest technical documentation of limitations and failure modes, including a real security incident (an exposed API key), rather than presenting only favourable results.
- Delivering a complete engineering system under a genuine zero-budget constraint.

[object Object] (

[YOUR NAME], to be personalised before submission):

- Most difficult engineering problem encountered and how it was solved: the temporal-encoding defect (Section 4.5) was the hardest to find because the system appeared to work correctly on simple cases and only failed on a specific combination of inputs; it was solved by writing a minimal, controlled reproduction script rather than reasoning about the code abstractly, which immediately revealed the Z3 arity mismatch.
- A key engineering decision made personally and the evidence behind it: choosing to cap automatic regeneration at two retries and disclose an unresolved contradiction, rather than looping indefinitely

or hiding the conflict, based on live evidence that some contradictions originate in the user's own message and are therefore not fixable by rephrasing the bot's reply.

- New knowledge acquired independently: how to read and interpret a Z3 unsat core to identify exactly which clauses are jointly responsible for an unsatisfiable formula, which is not something covered in standard coursework.
- What would be redesigned if repeated: predicate identity would be based on a normalised, semantically-matched representation from the outset, rather than discovering through benchmarking that exact-name matching is the system's principal weakness.

REFERENCES

[1] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” in Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Lecture Notes in Computer Science, vol. 4963, Springer, 2008, pp. 337–340.

[2] Ollama Inc., “Ollama: Get up and running with large language models locally,” [Online]. Available: https://ollama.com. [Accessed: 2026].

[3] Qwen Team, Alibaba Group, “Qwen2.5 Technical Report,” 2024. [Online]. Available: https://qwenlm.github.io.

[4] FastAPI, “FastAPI: high-performance web framework for building APIs with Python,” [Online]. Available: https://fastapi.tiangolo.com.

[5] A. Hagberg, D. Schult, and P. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in Proc. 7th Python in Science Conf. (SciPy), 2008, pp. 11–15.

[6] Meta AI, “Llama 3.1 Model Card,” 2024. [Online]. Available: https://ai.meta.com/llama.

[7] Anthropic, “Claude,” AI assistant used as a code-generation and debugging aid during development (disclosed per the Declaration, page iii). [Online]. Available: https://claude.ai. [Accessed: 2026].

[8] React, “React – A JavaScript library for building user interfaces,” [Online]. Available: https://react.dev.

[9] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2015, pp. 632–642.

[10] Internet Engineering Task Force (IETF), “RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format,” 2017.

[11] OWASP Foundation, “OWASP API Security Top 10,” [Online]. Available: https://owasp.org/API-Security.

APPENDICES

Appendix A – Detailed Calculations

Not applicable in the traditional numerical sense: this is a software project with no physical-quantity derivations. The project's single essential “calculation” — the formal contradiction/entailment/consistency decision procedure — is given in full in Section 3.5 and is not repeated here.

Appendix B – Engineering Drawings / Schematics

Figures 1, 2, and 4 of this report (Sections 3.5 and 4.4) serve as the system's architecture diagram, control-flow schematic, and proof-graph example respectively; no additional physical drawings apply to a software system.

Appendix C – Source Code / Code Repository Information

Only an essential excerpt is included below; the complete, runnable source code is maintained in the project repository, organised as core/, parser/, engine/, chat/, graph/, storage/, api_ui/, frontend/, benchmark/, and tests/.

```
def check(self, kb, turn_id, turn_text, extraction):     prior_entries = list(kb.entries)
new_entries = kb.register_extraction(turn_id, turn_text, extraction)     all_entries =
prior_entries + new_entries     solver = kb.build_solver(all_entries)     if solver.check() ==
z3.unsat:         core_entries = [e for e in all_entries if str(e.tracker) in
{str(c) for c in solver.unsat_core()}]         return CONTRADICTION, core_entries     # ...
entailment check (KB ^ not-turn) follows, see engine/consistency.py
```

Appendix D – Datasheets

Not applicable — no physical hardware components were used.

Appendix E – Experimental / Raw Data

The complete raw benchmark output (per-case gold label, system label, extracted symbolic rendering, and latency) is retained as a JSON report in benchmark/reports/ within the project repository, generated by benchmark/run_benchmark.py.

Appendix F – Ethics / Institutional Approval

Not applicable — this software-only project involved no human subjects, animal subjects, or physical experimentation requiring institutional ethics approval.

Appendix G – Project Meeting / Progress Records

Development proceeded as a continuous, iterative solo effort; no formal meeting minutes were recorded. Section 4.8 (Gantt chart, Figure 3) serves as the milestone record for the project timeline.

Appendix H – User / Industry Feedback

Not applicable for this submission.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable for this submission.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (page iv) and Section 4.8: this was a solo project with 100% individual contribution across requirements, design, implementation, testing, and documentation.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2 / SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2 / SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1 / SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)